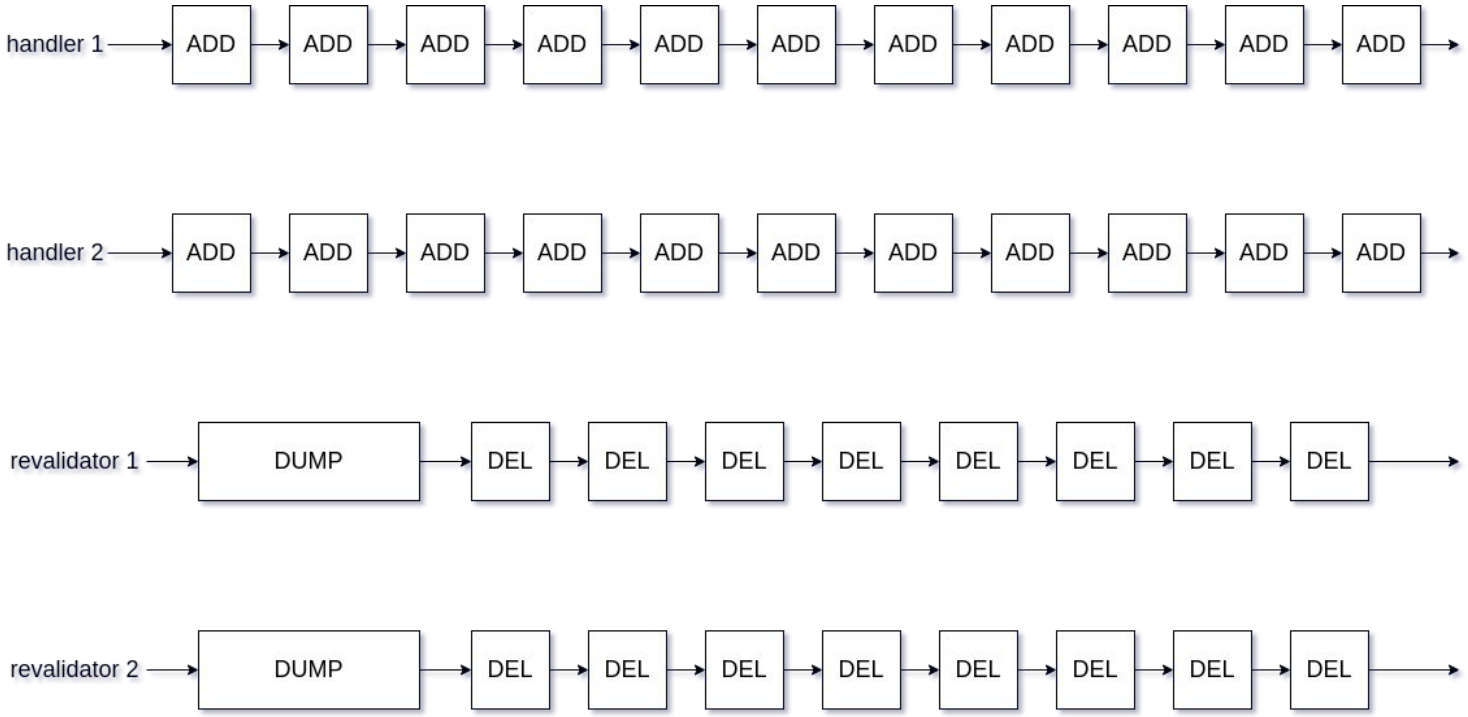
Where turbo boosting TC flower control path had led us to

Marcelo Ricardo Leitner, Vlad Buslov
Red Hat, Nvidia
mleitner@redhat.com, vladbu@nvidia.com

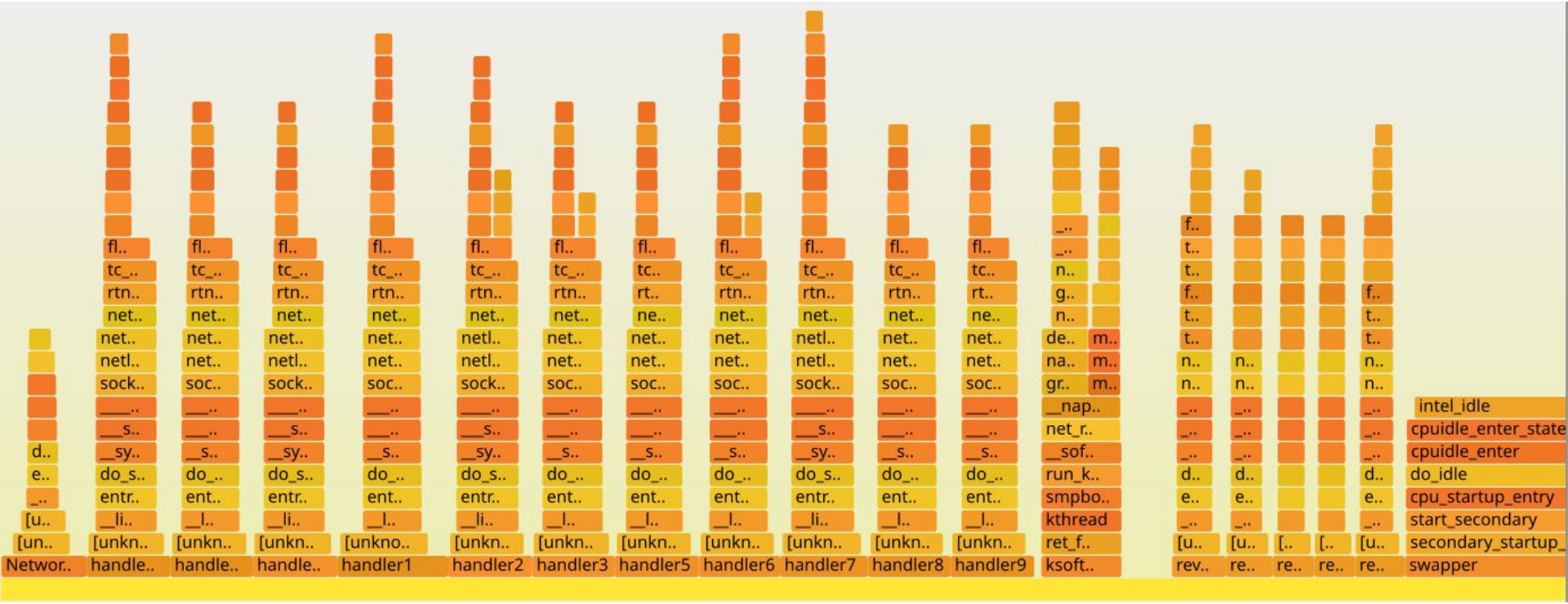
Agenda

- Introduction into OvS threading model
- Recapitulate TC changes we did in recent years on this matter
- Analyze how they impacted OvS learning rate
- Provide a current situation and open points, from TC perspective

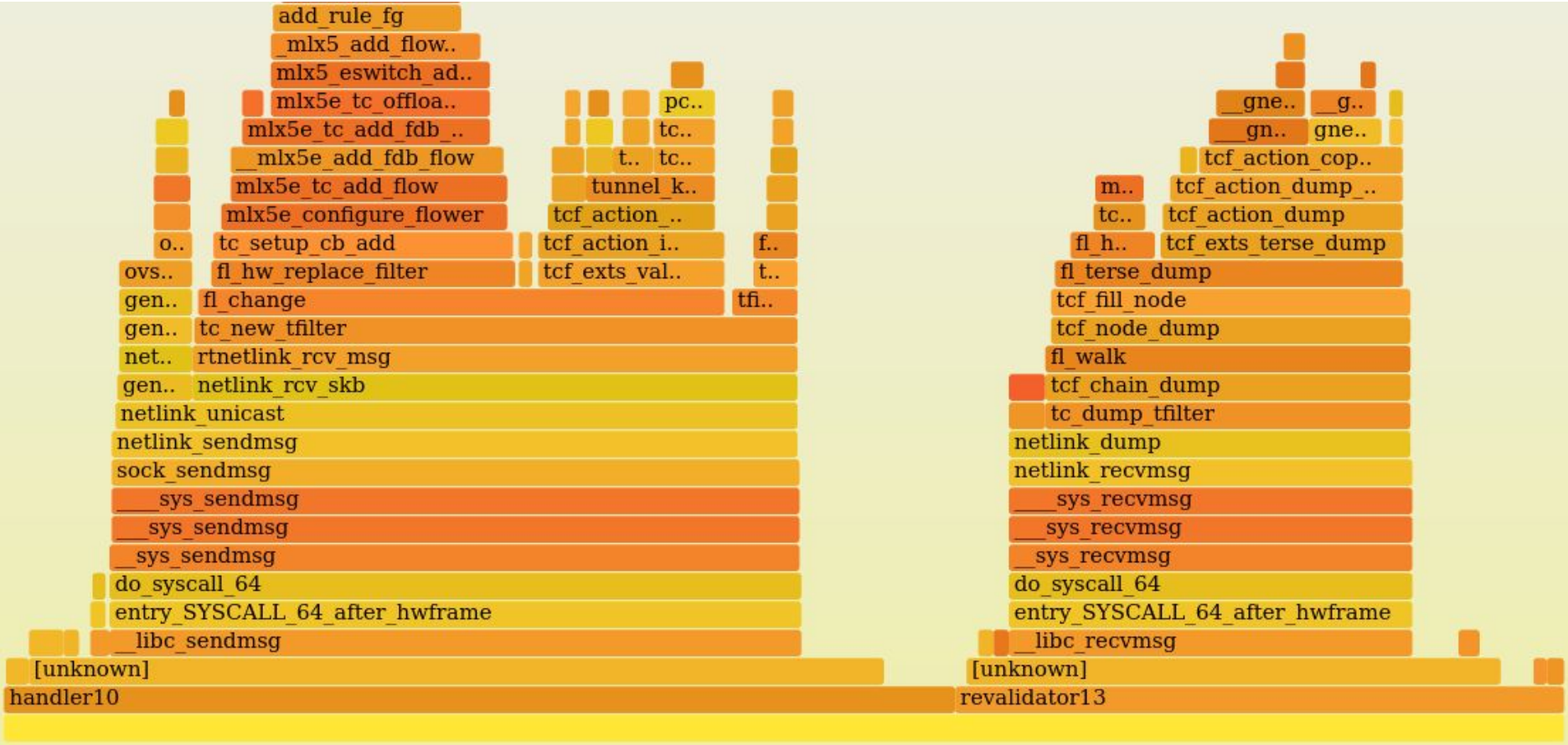
OvS threading model - idealized



OvS threading model - quick real world view



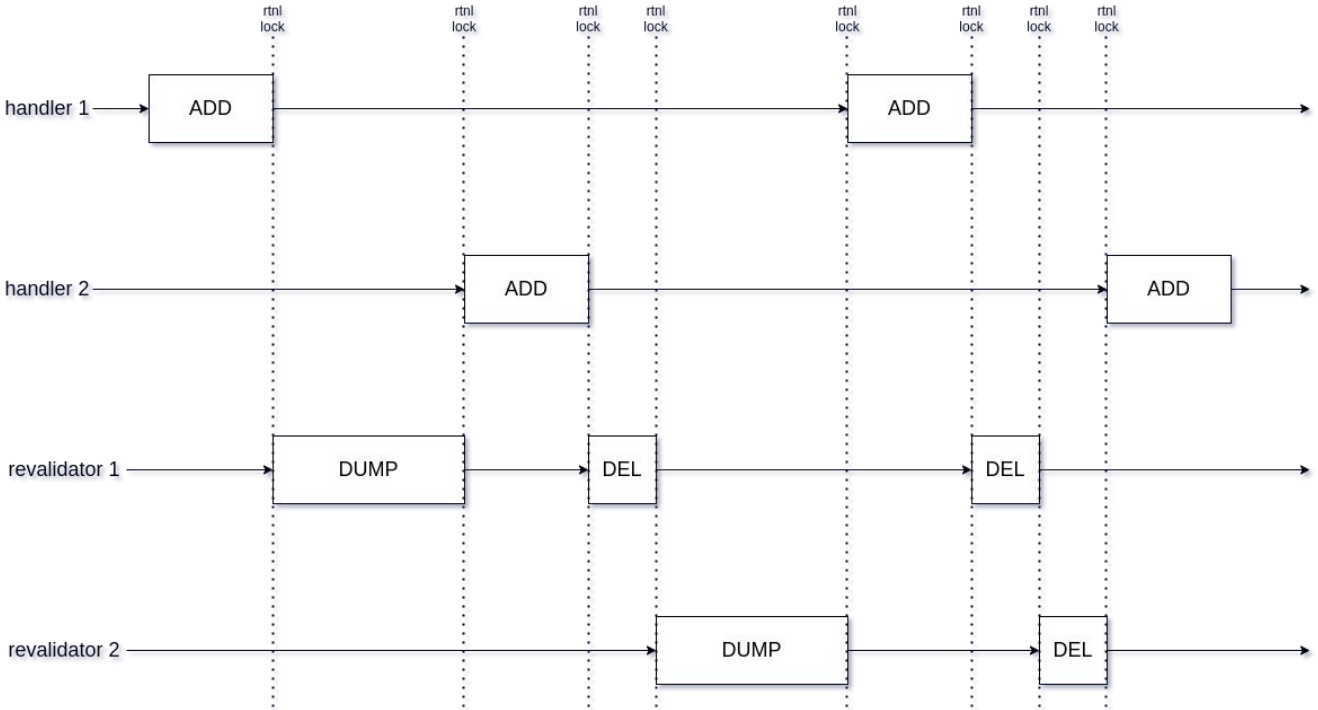
OvS threading model - quick real world view



Issues with TC performance

- Scalability - initially both classifier and action APIs exhibited quadratic behaviours in several places.
- Parallelism - one-big-lock rtnl_lock synchronizes everything (and is shared with other subsystems).
- Performance - several bottlenecks and inefficiencies for OvS hardware offload use-case in TC and mlx5 steering.

OvS threading model - how it was with tc



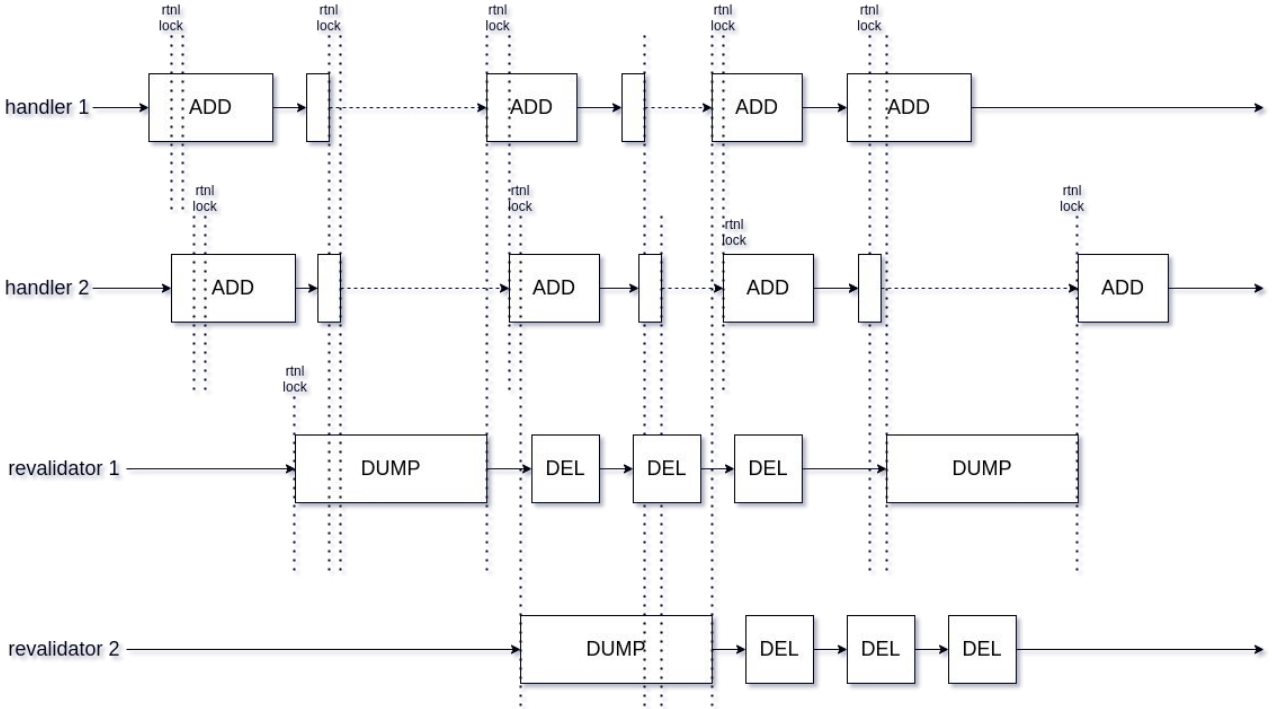
Scalability

- Convert cls_flower to use idr instead of linked list for managing filters (4.14).
- Refactor action API to use idr instead of hash table with fixed number of buckets (4.14).
- Refactor fl_walk (flower classifier) to look up the previous position from cookie with idr instead of linear search in linked list (4.19).

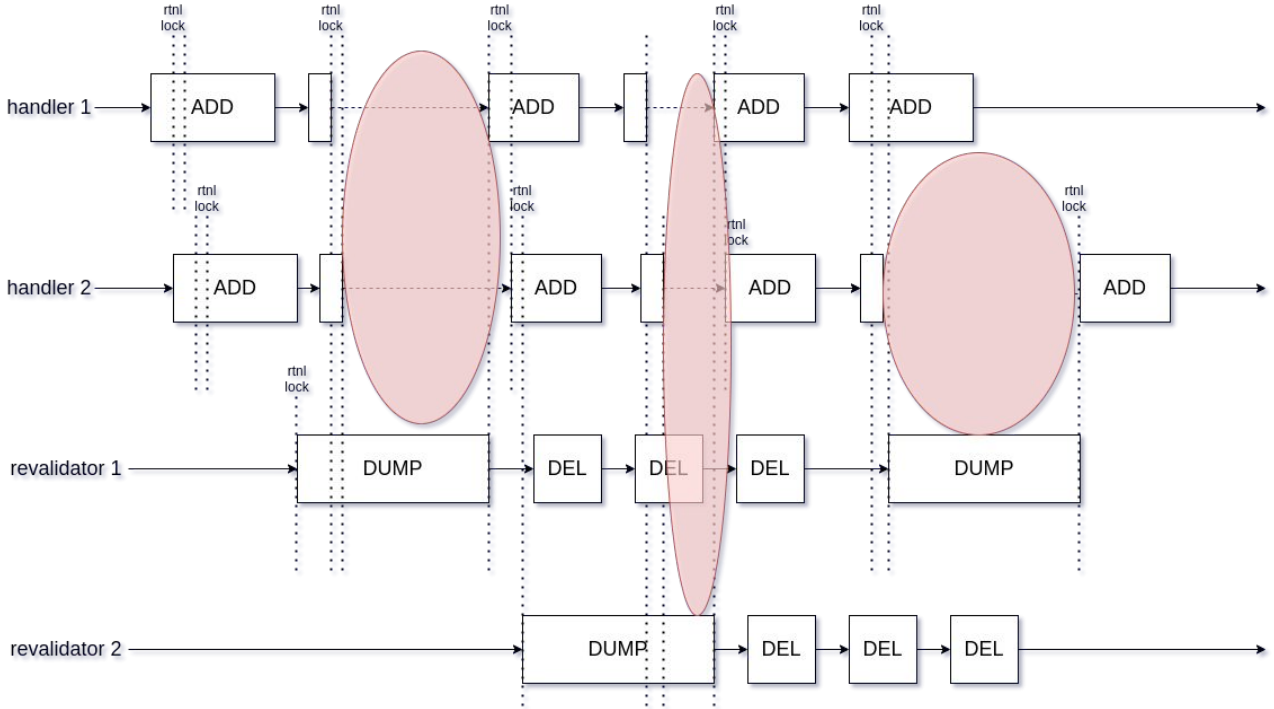
Parallelism

- Rtnetlink infrastructure for unlocked handlers (4.14)
- Unlocked action API (4.19)
- Unlocked actions (4.19)
- Lockless ingress Qdisc handling (4.20)
- Unlocked classifier API (5.1)
- Unlocked flower classifier (5.2)
- Unlocked mlx5 TC (5.4)
- Unlocked TC driver callbacks (5.4)

OvS with “almost” unlocked TC filter update



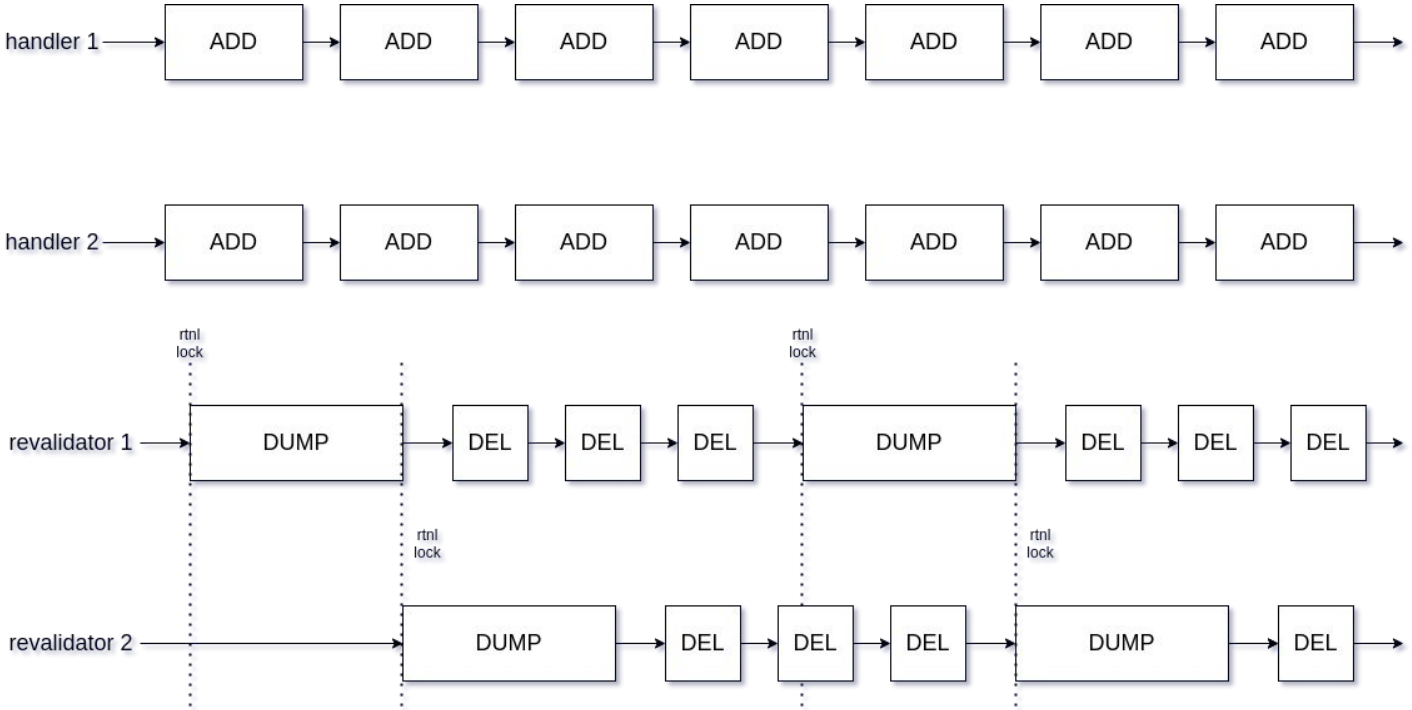
OvS with “almost” unlocked TC filter update (continued)



Parallelism (continued)

- Rtnetlink infrastructure for unlocked handlers (4.14)
- Unlocked action API (4.19)
- Unlocked actions (4.19)
- Lockless ingress Qdisc handling (4.20)
- Unlocked classifier API (5.1)
- Unlocked flower classifier (5.2)
- Unlocked mlx5 TC (5.4)
- Unlocked TC driver callbacks (5.4)
- Unlocked flow_action infrastructure (5.7)

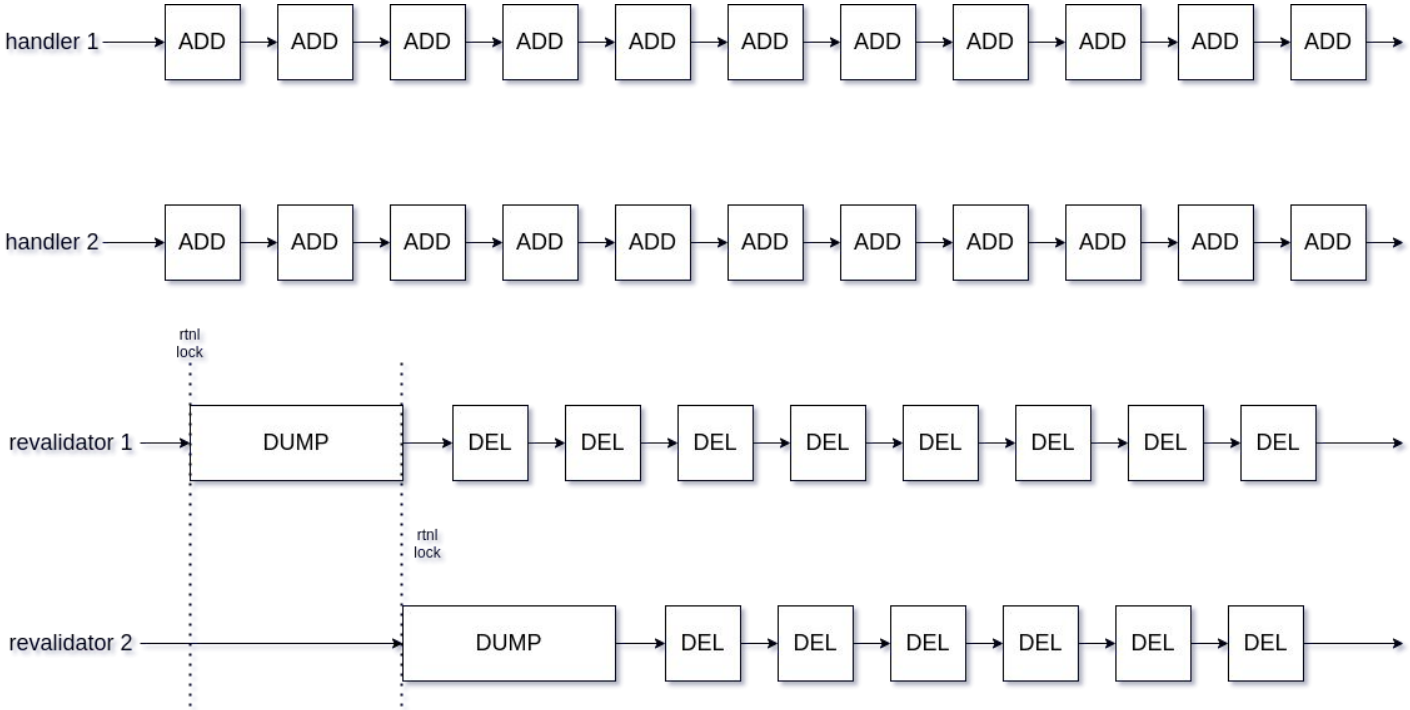
OvS with unlocked TC filter update



Performance

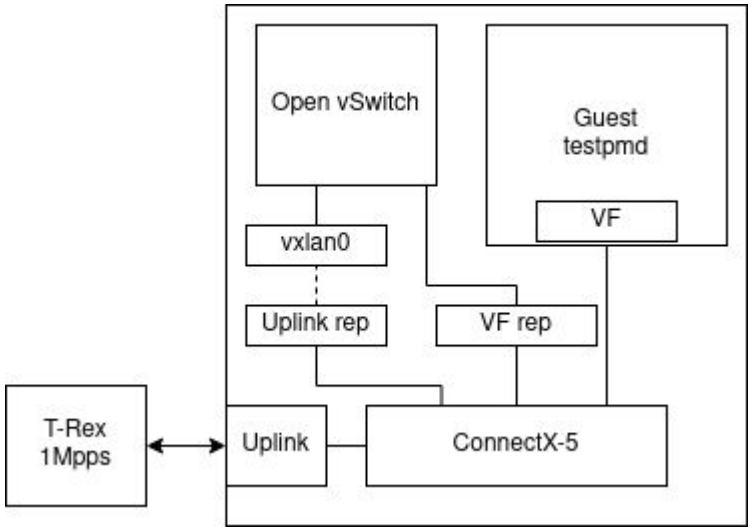
- Mlx5 software steering (5.4-ongoing).
- Don’t use percpu stats in actions (5.5).
- Filter/action terse dump (5.8)

OvS threading model - how it is with tc

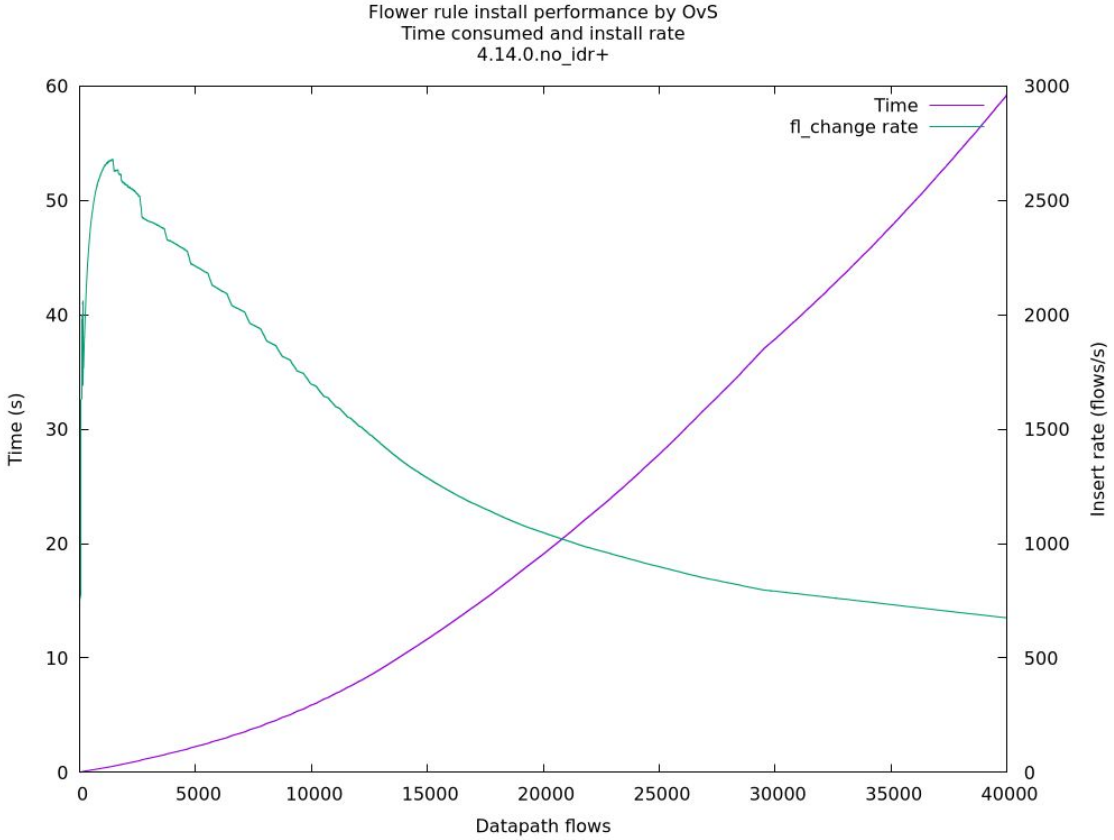


Test setup

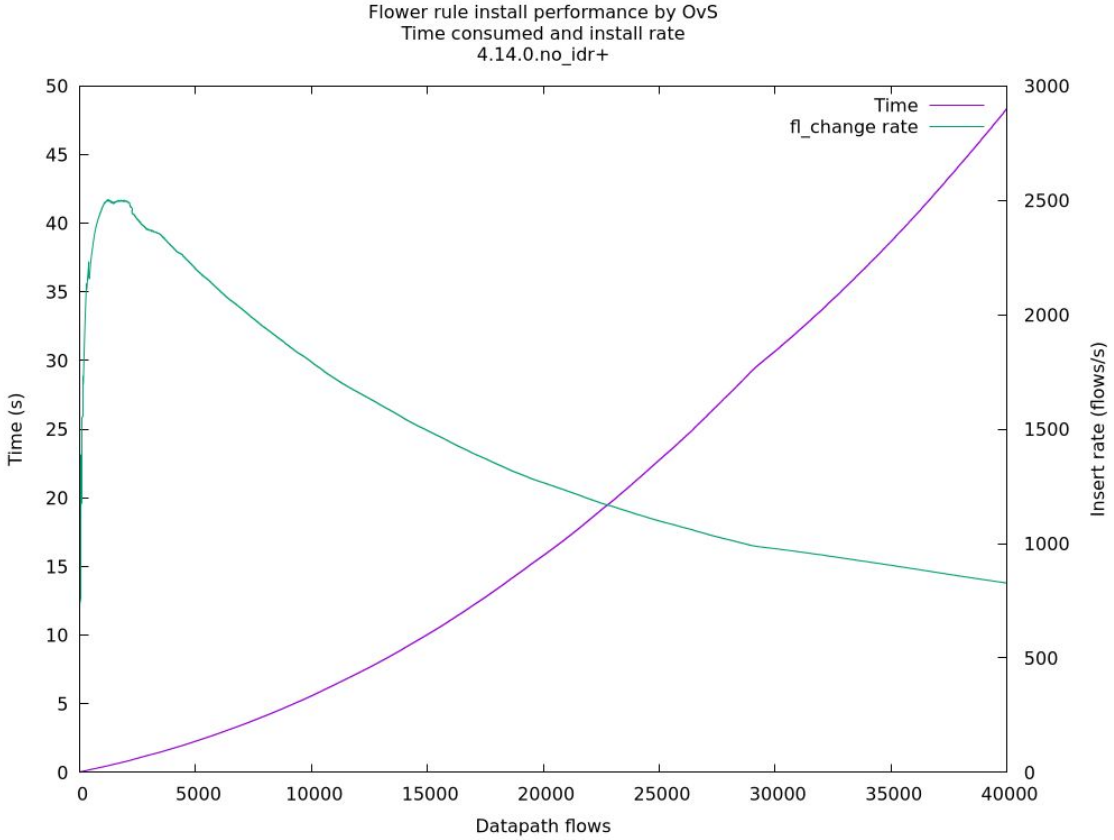
- OvS 2.15, with terse dumps and no_percpu
- VXLAN, small UDP packets inside
- Varying inner src MAC address
- testpmd in macswap mode
- 2 datapath flows per T-Rex flow



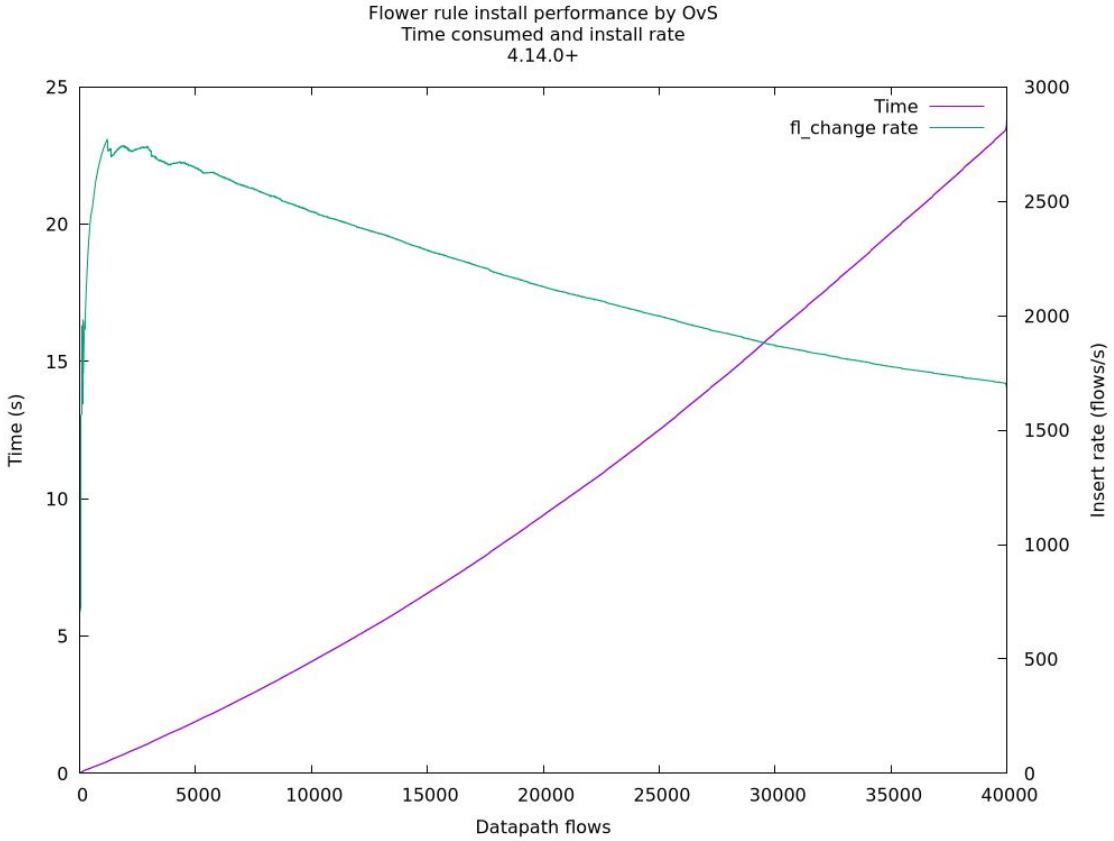
v4.14 without idr patches - 4 CPUs on host



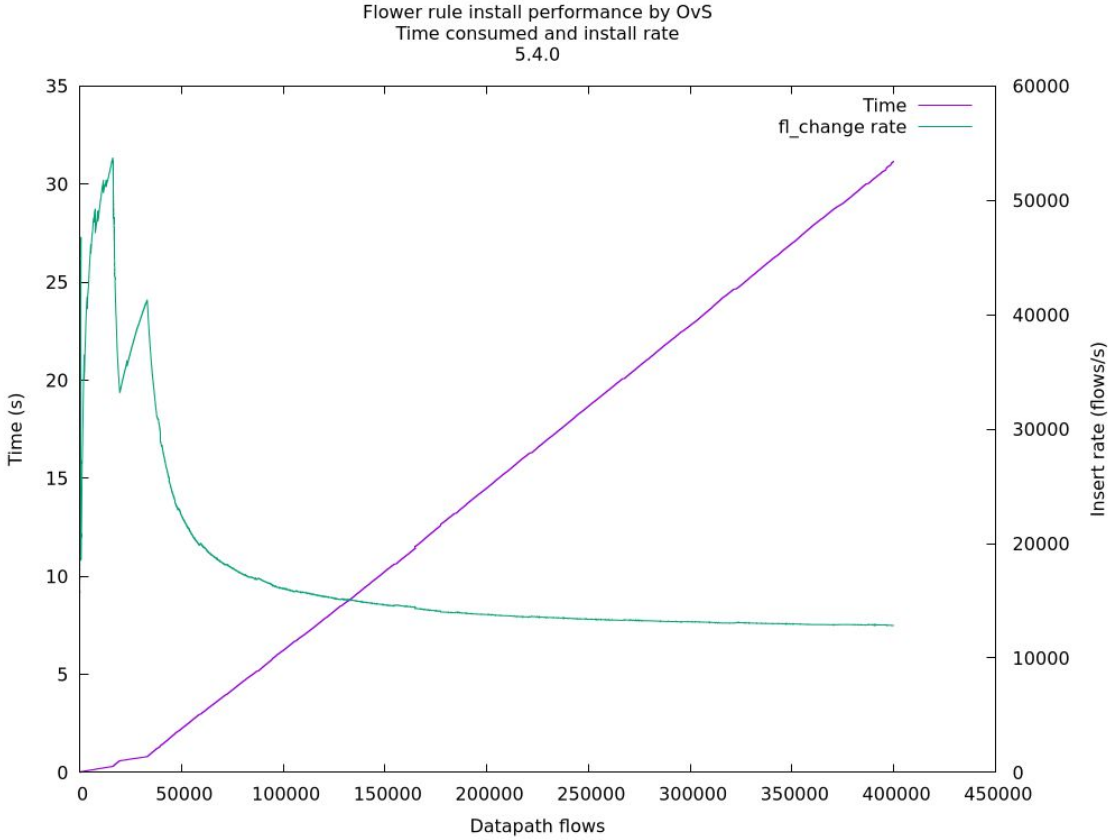
v4.14 without idr patches - 16 CPUs on host



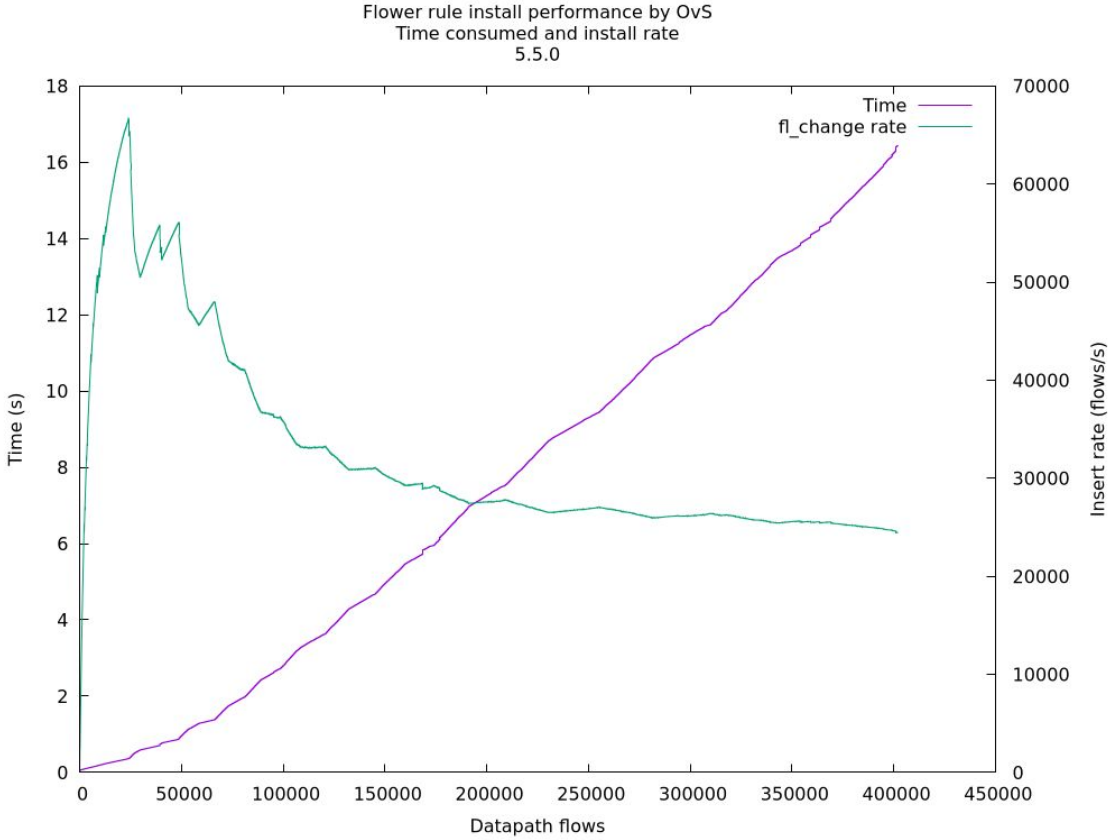
v4.14 with idr patches - 16 CPUs on host



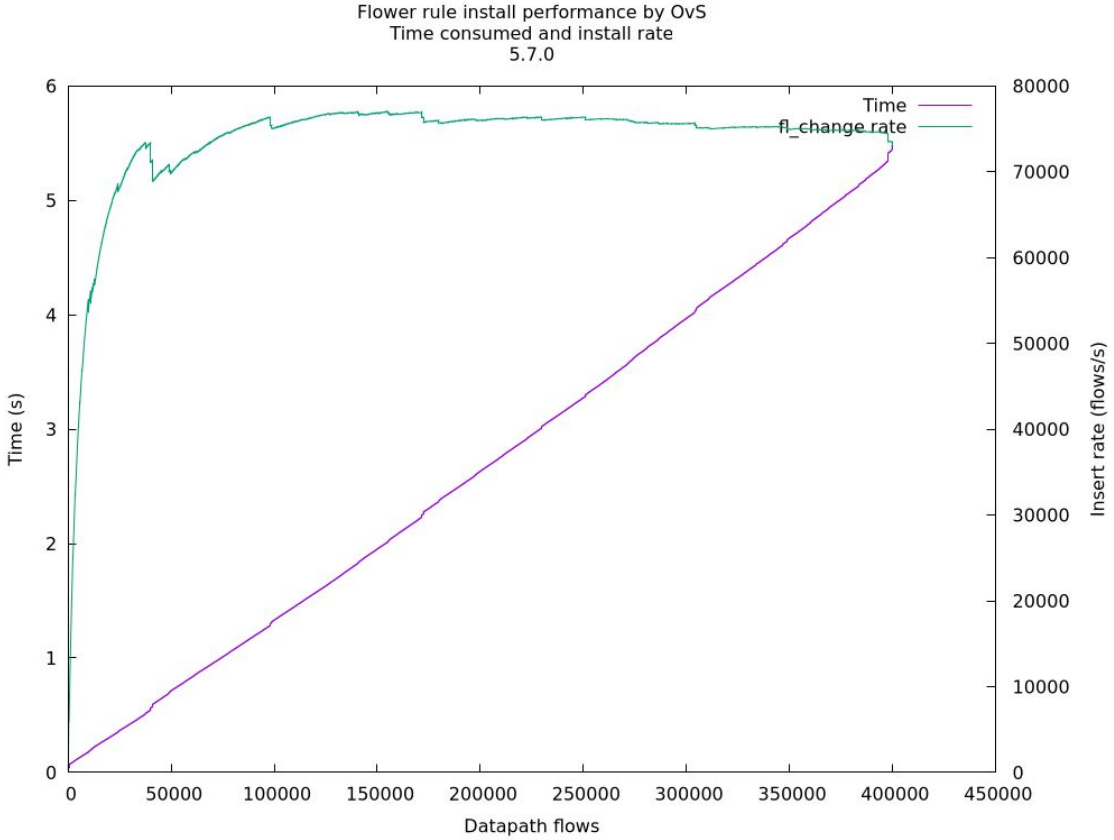
v5.4 - 16 CPUs, TC unlock and initial SW steering



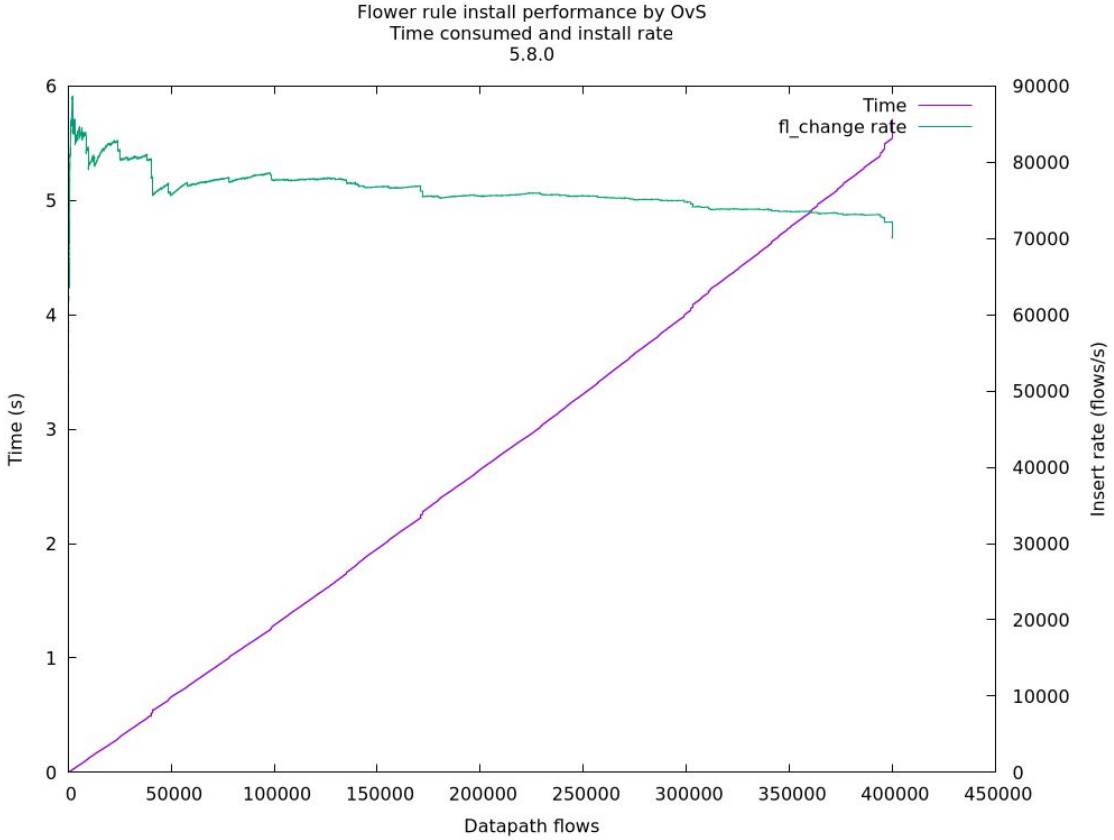
v5.5 - 16 CPUs on host, no percpu



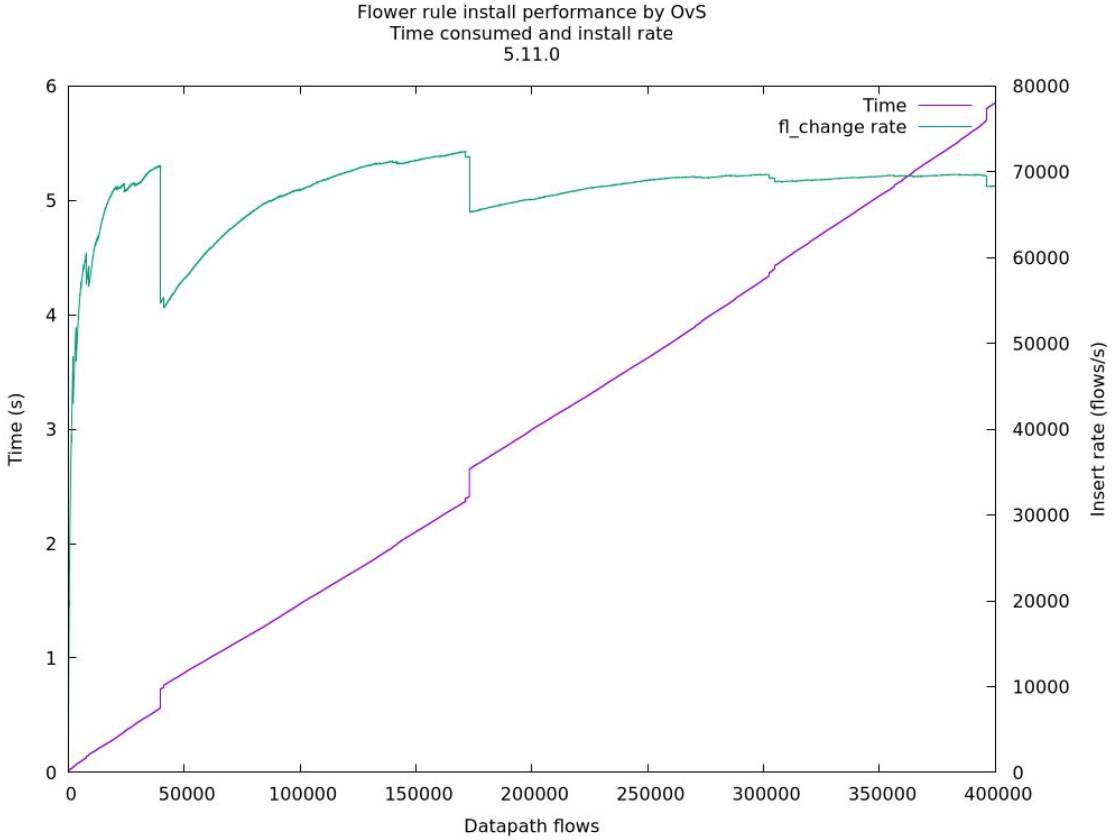
v5.7 - 16 CPUs on host, flow_action unlock



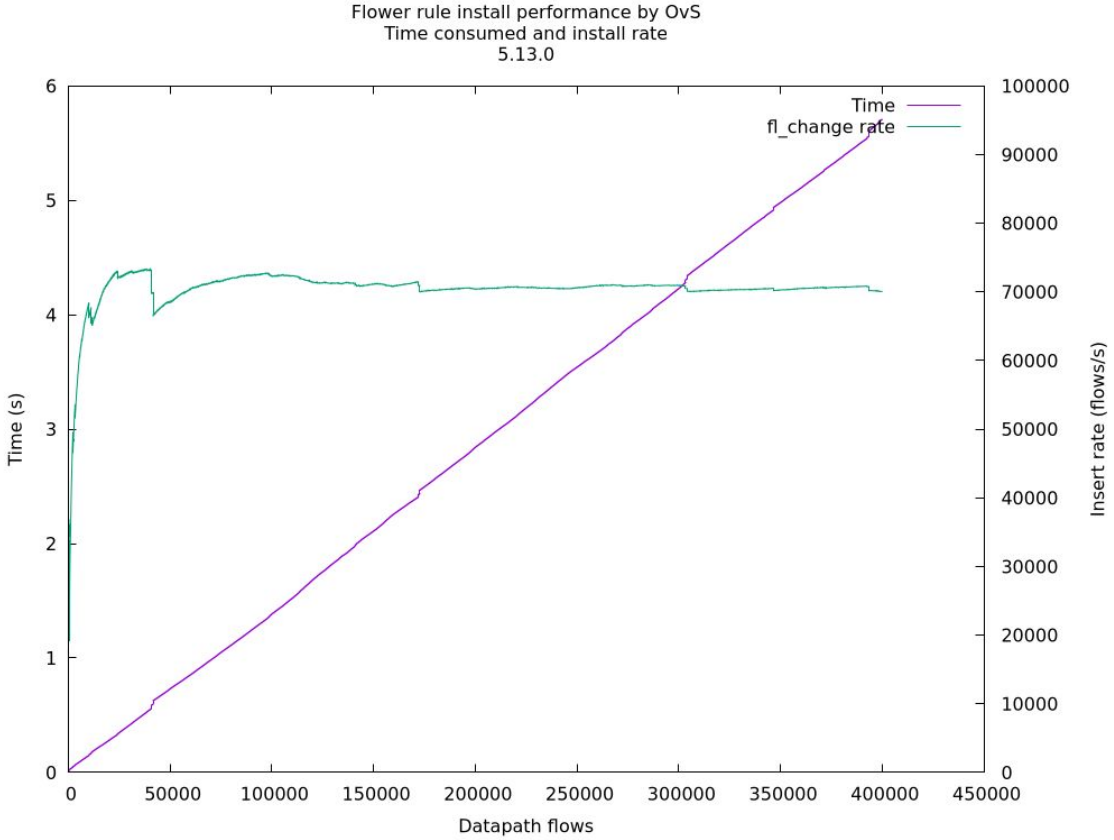
v5.8 - 16 CPUs on host, terse dumps



v5.11 - 16 CPUs, stable SW steering with buddy allocator



v5.13 - 16 CPUs on host



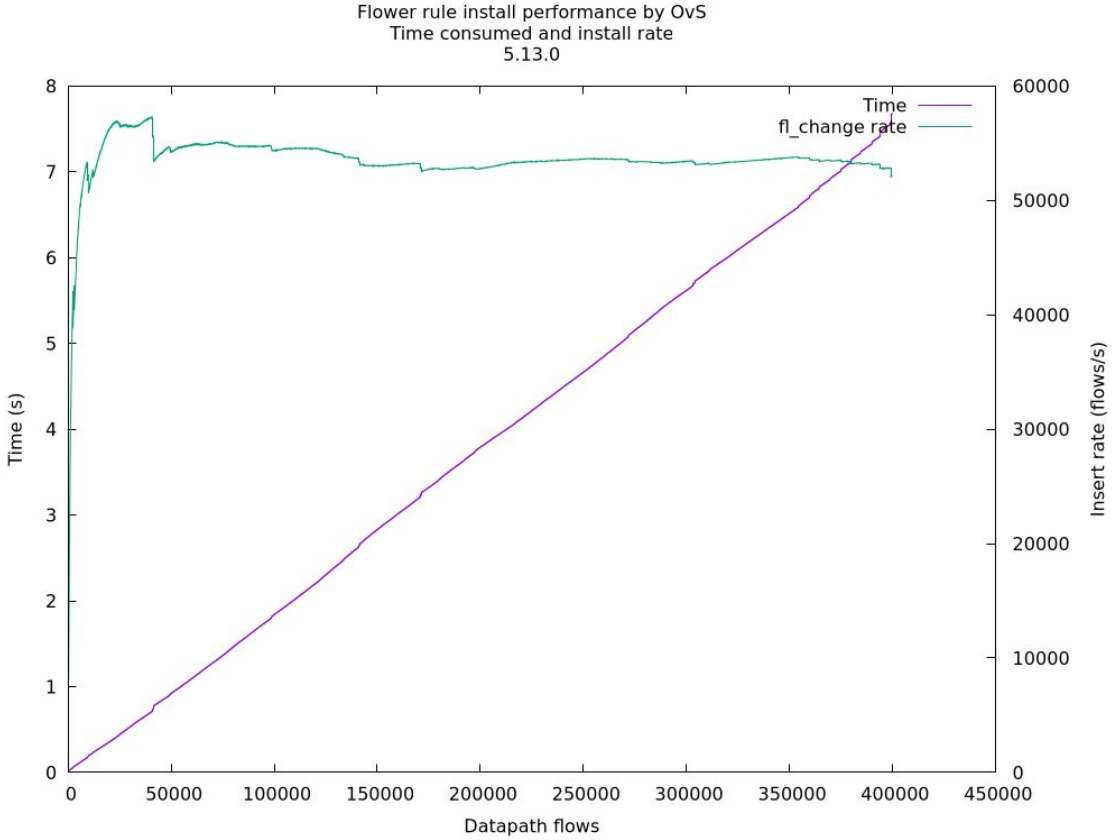
87x faster

than

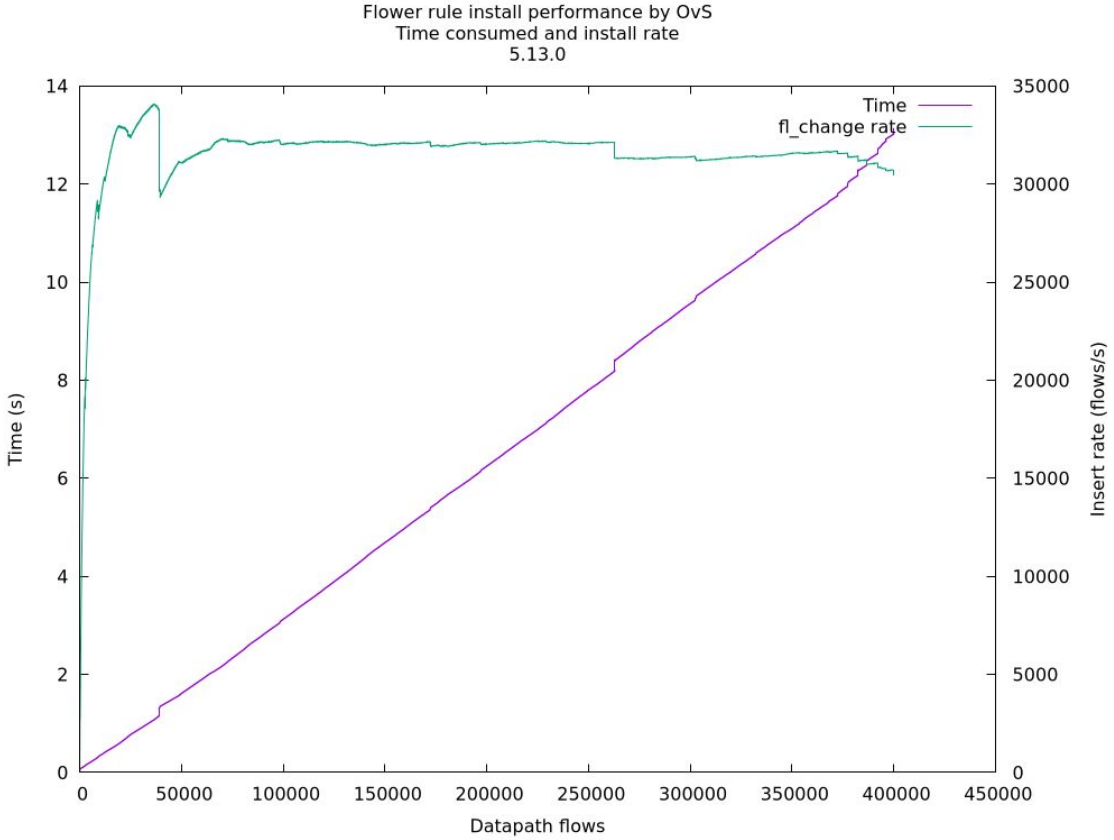
v4.14 without

idr patches

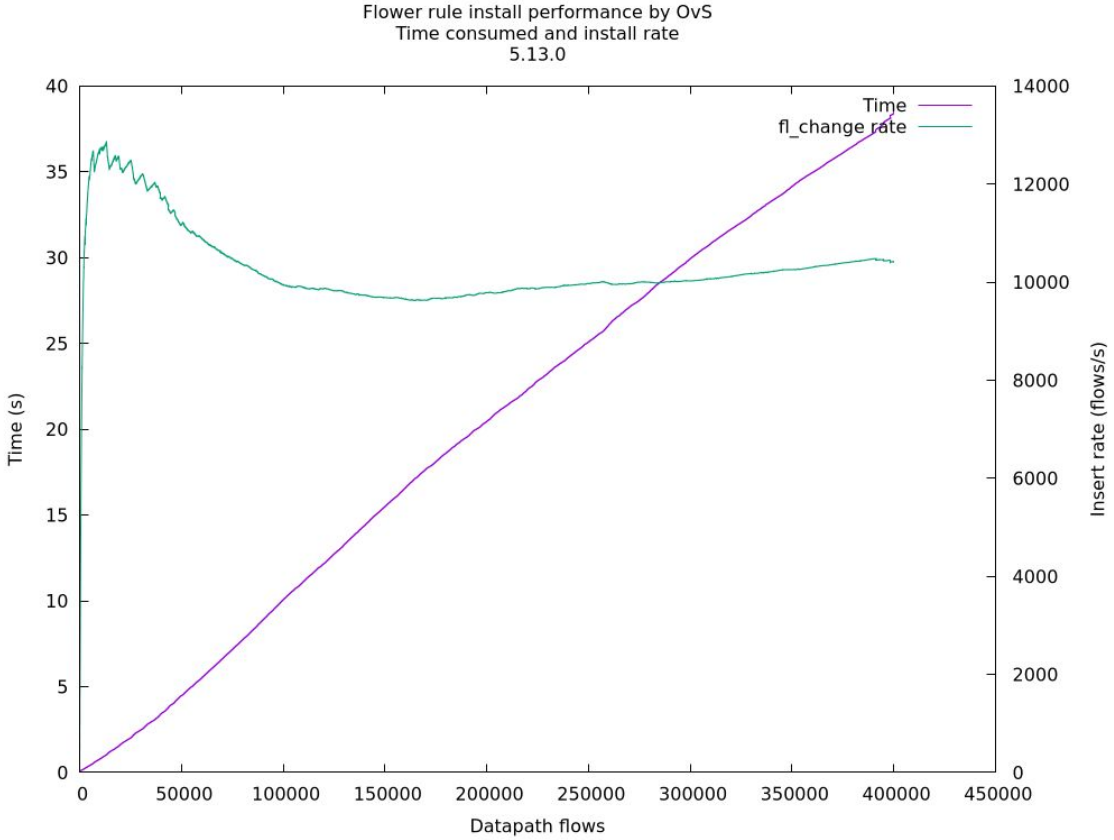
v5.13 - 12 CPUs on host



v5.13 - 8 CPUs on host



v5.13 - 4 CPUs on host



15x faster

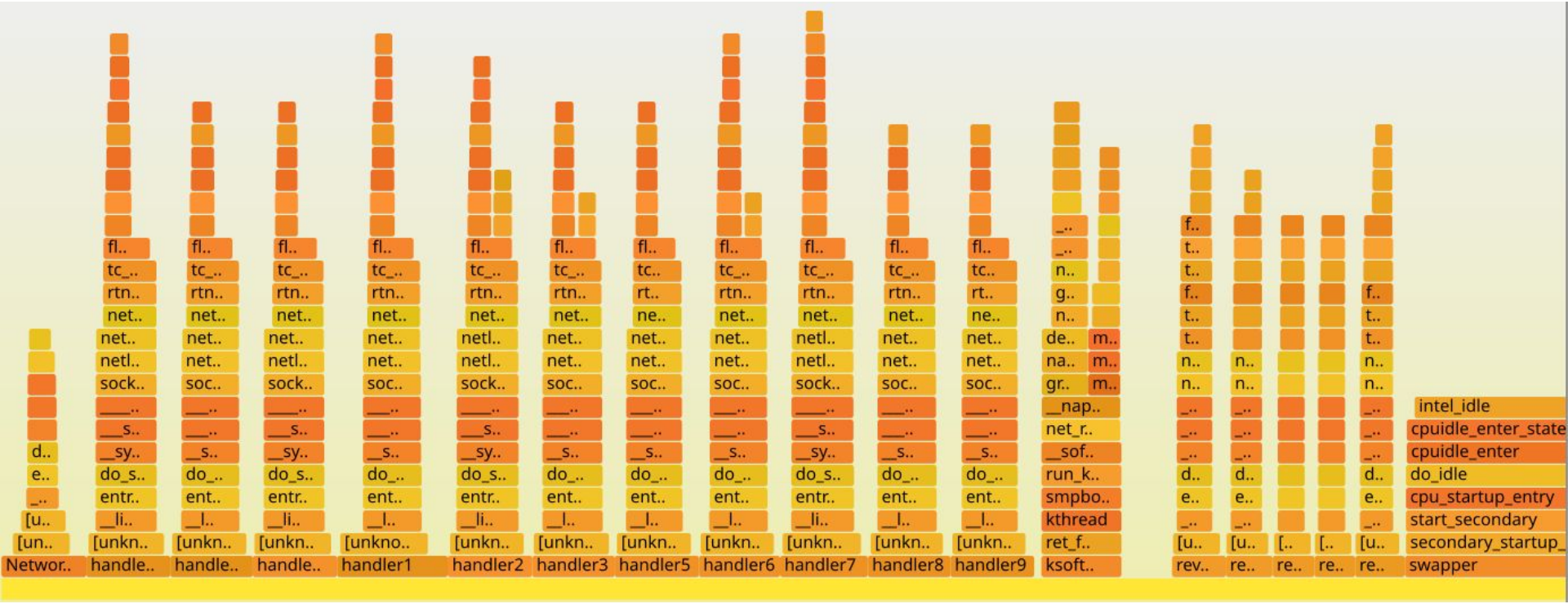
than

v4.14 without

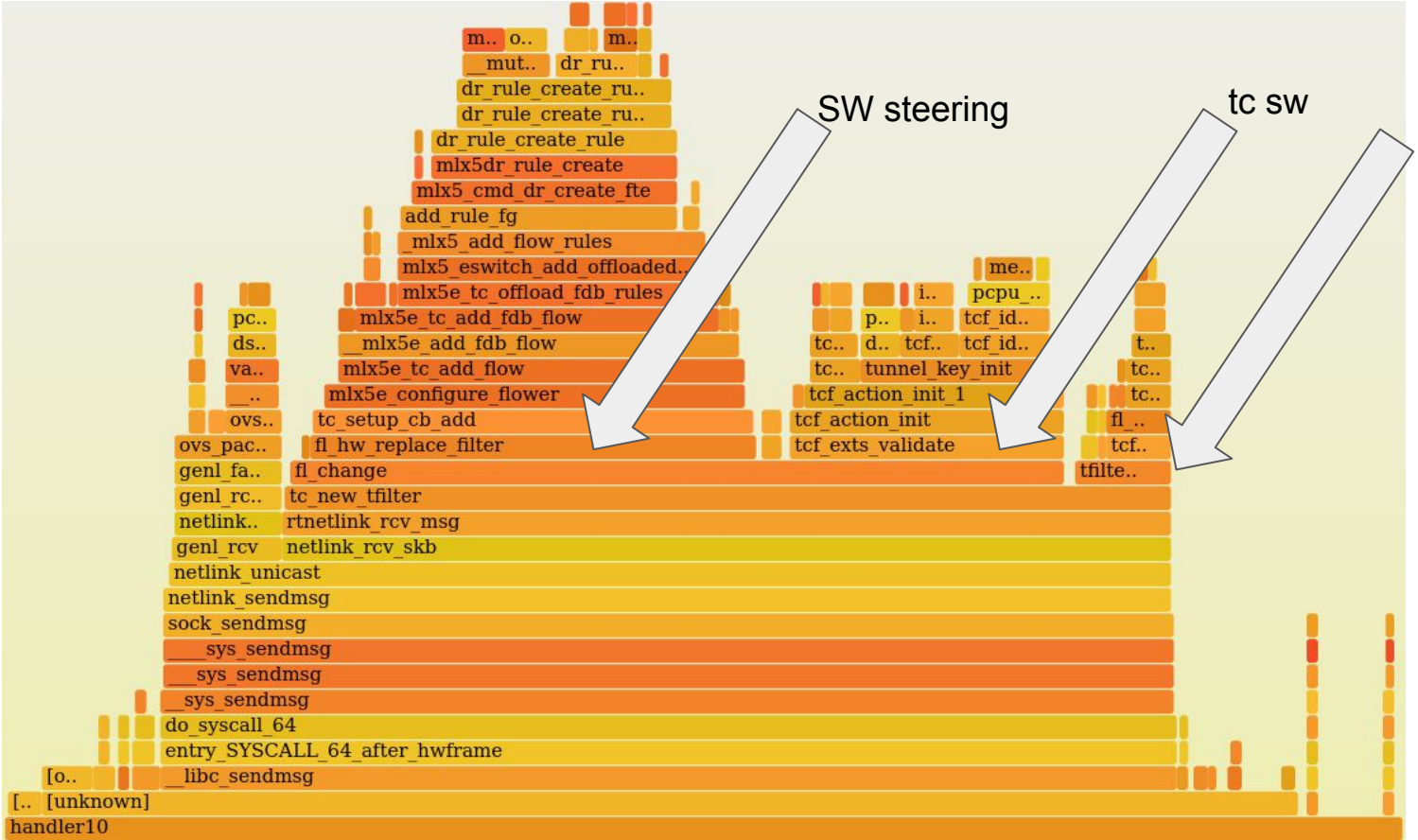
idr patches

Hopefully you refilled your coffee,
because it's….
flamecharts time!

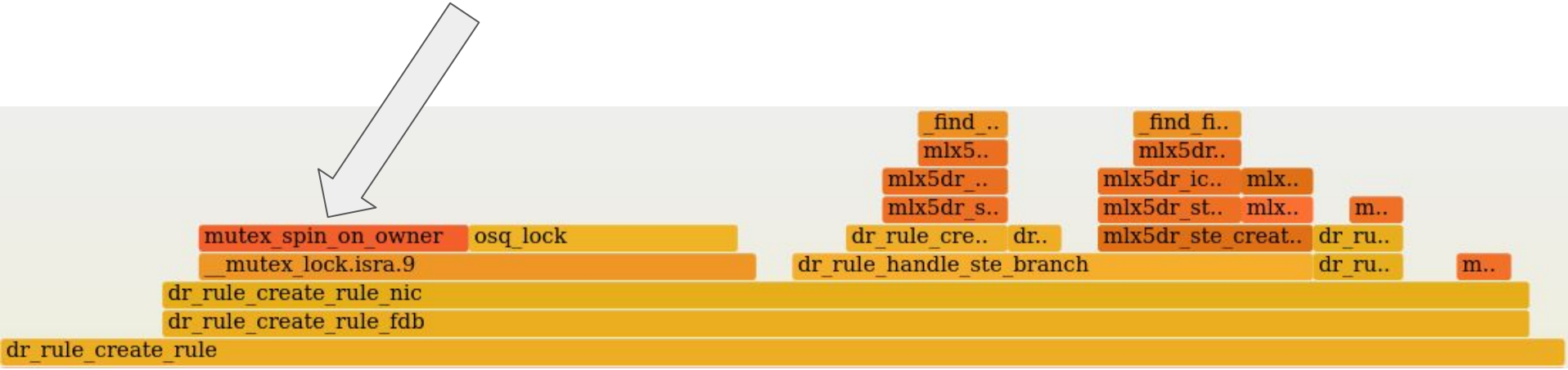
v5.13 - overall (16) CPU usage



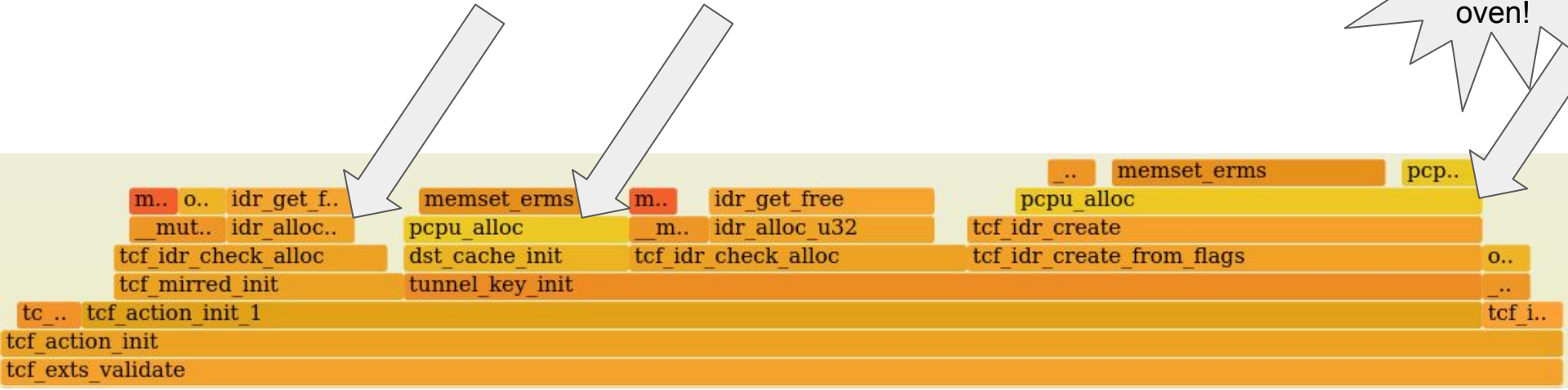
v5.13 - handler thread



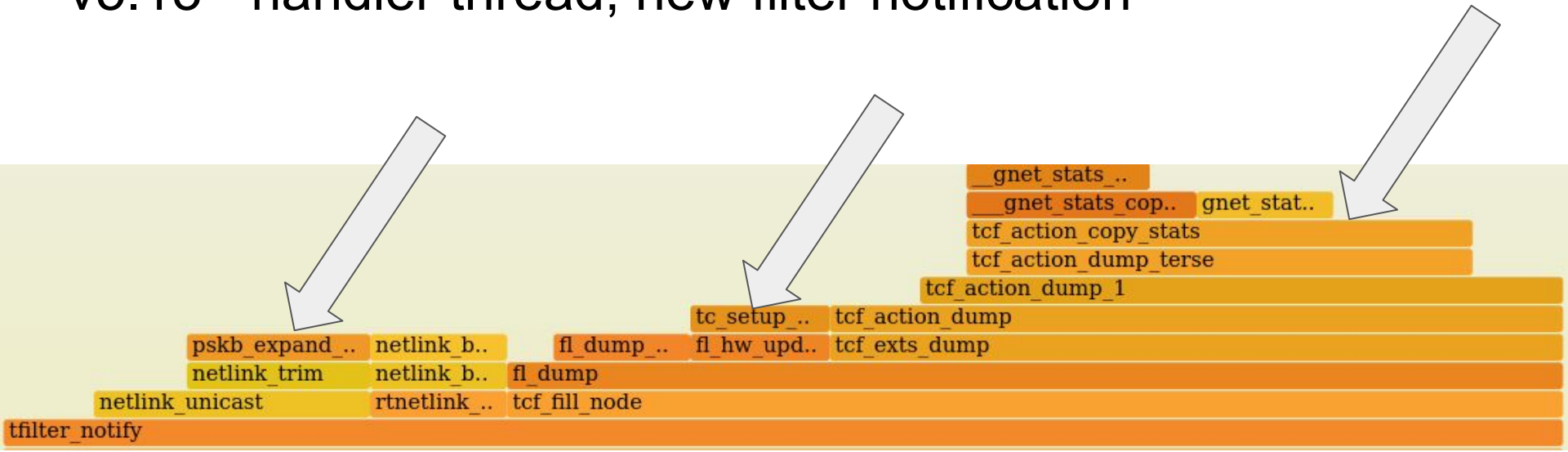
v5.13 - handler thread, mlx5 sw steering lock contention



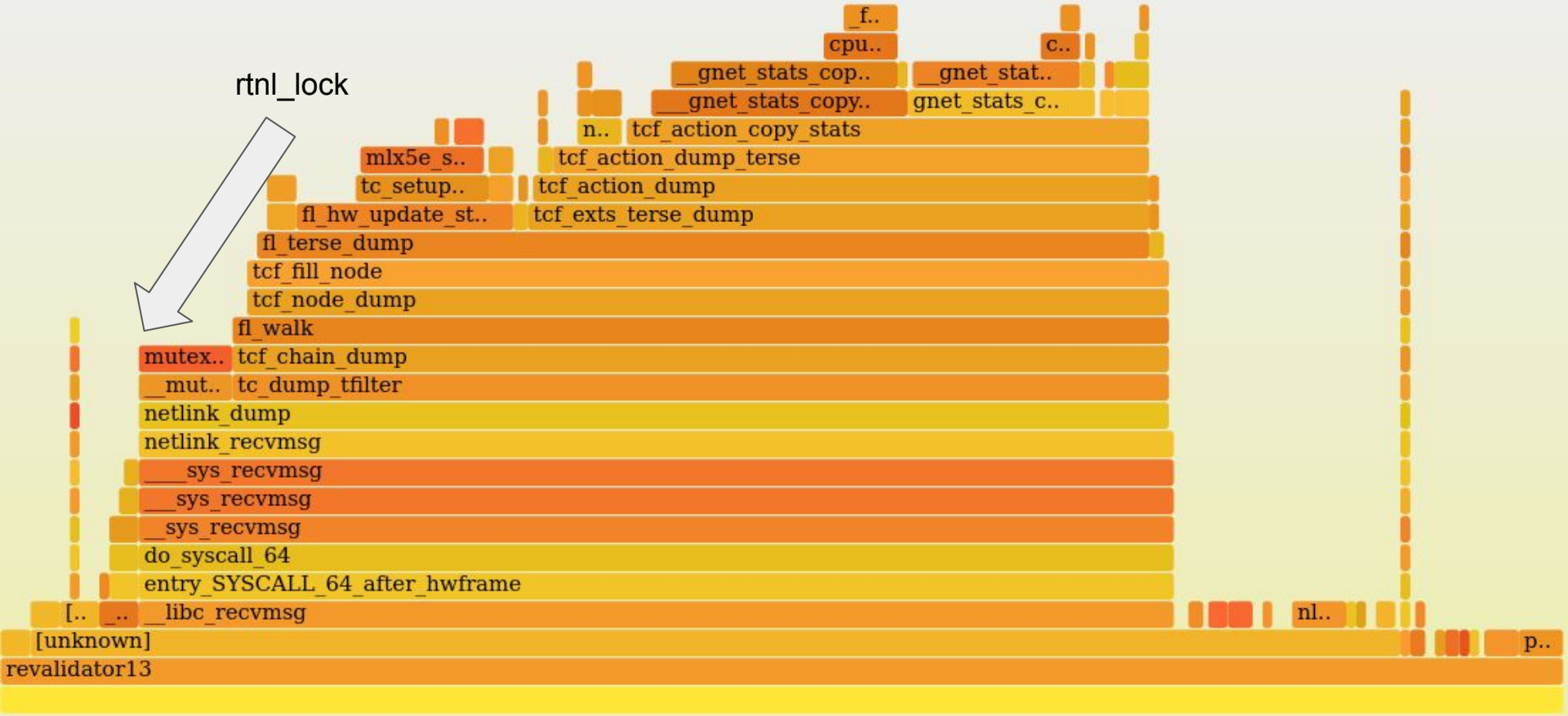
v5.13 - handler thread, idr and percpu allocs



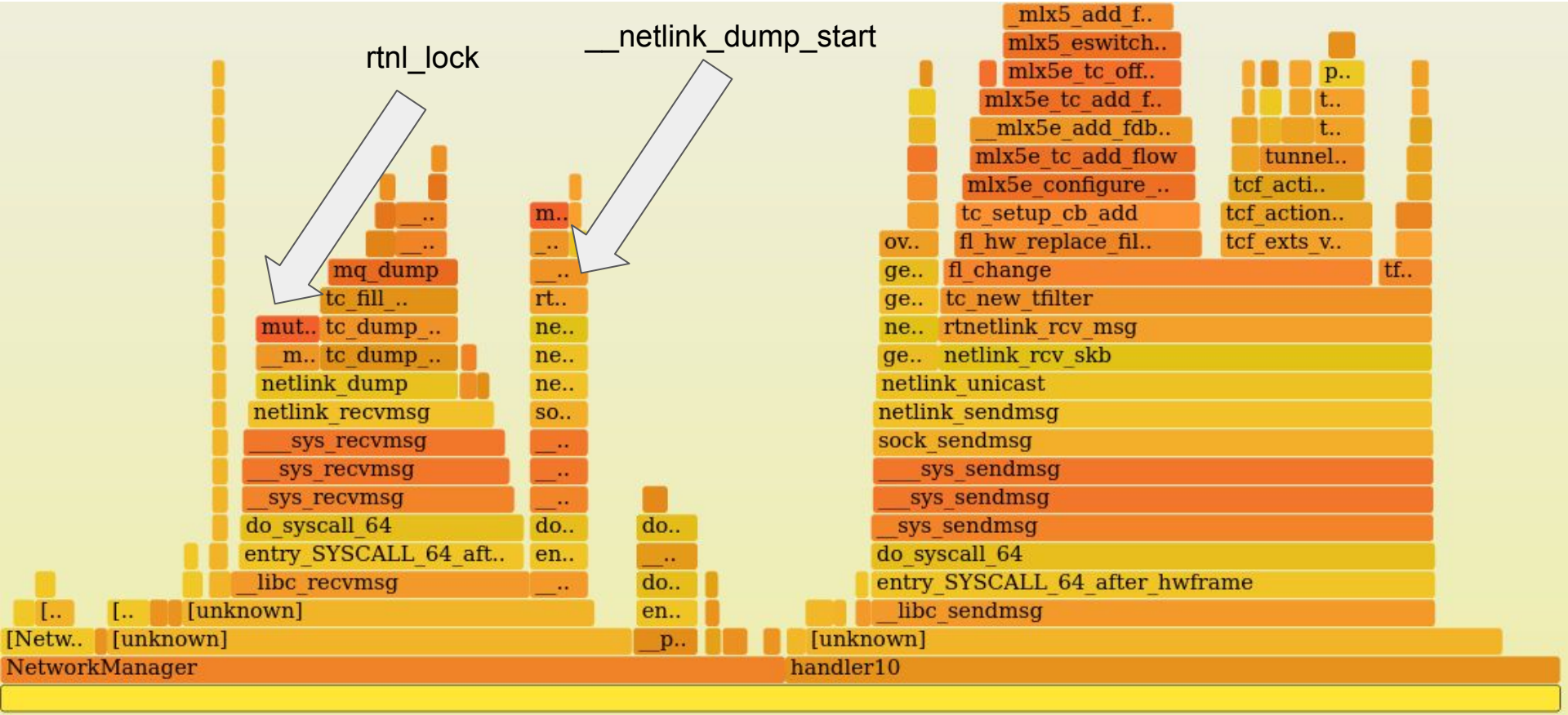
v5.13 - handler thread, new filter notification



v5.13 - revalidator thread, just stats (no deletion)



v5.13 - NM and a handler thread



Summary and conclusions

- Some fixes that removed contention also sped it up (idr, no_percpu)
- More stable and scalable
- 15x improvement with 4 CPUs and 87x with 16 CPUs
- OvS specific aspects were not covered here and they may also help
- Still some open points
  - idr and radix tree
  - percpu used by tunnel_key indirectly through dst_cache
  - filter update notifications
    - realloc in there is a lower hanging fruit
    - interaction with NetworkManager & others
  - sw steering

Thanks!